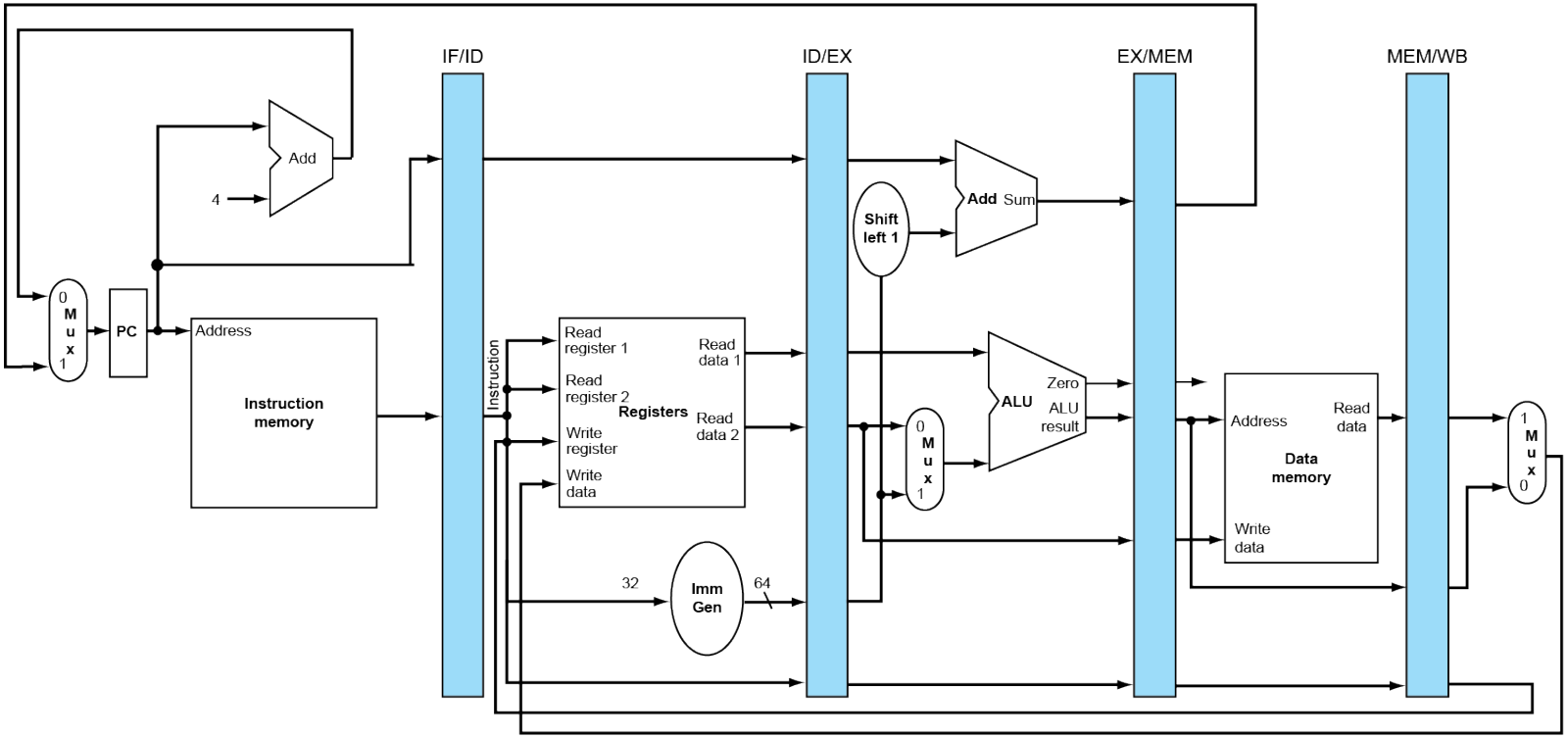
Chapter 4

The Processor

Revision:Single-Cycle Diag

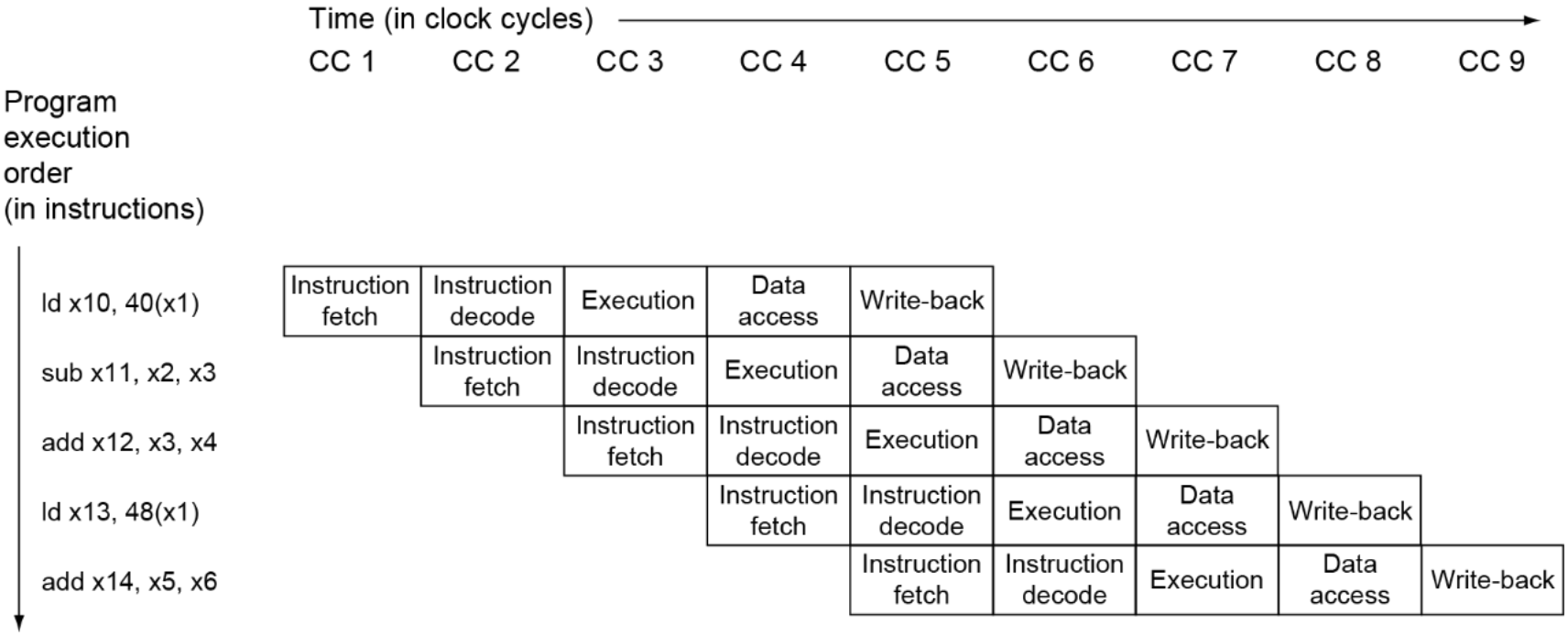
- State of pipeline in a given cycle

add x14, x5, x6	ld x13, 48(x1)	add x12, x3, x4	sub x11, x2, x3	ld x10, 40(x1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back

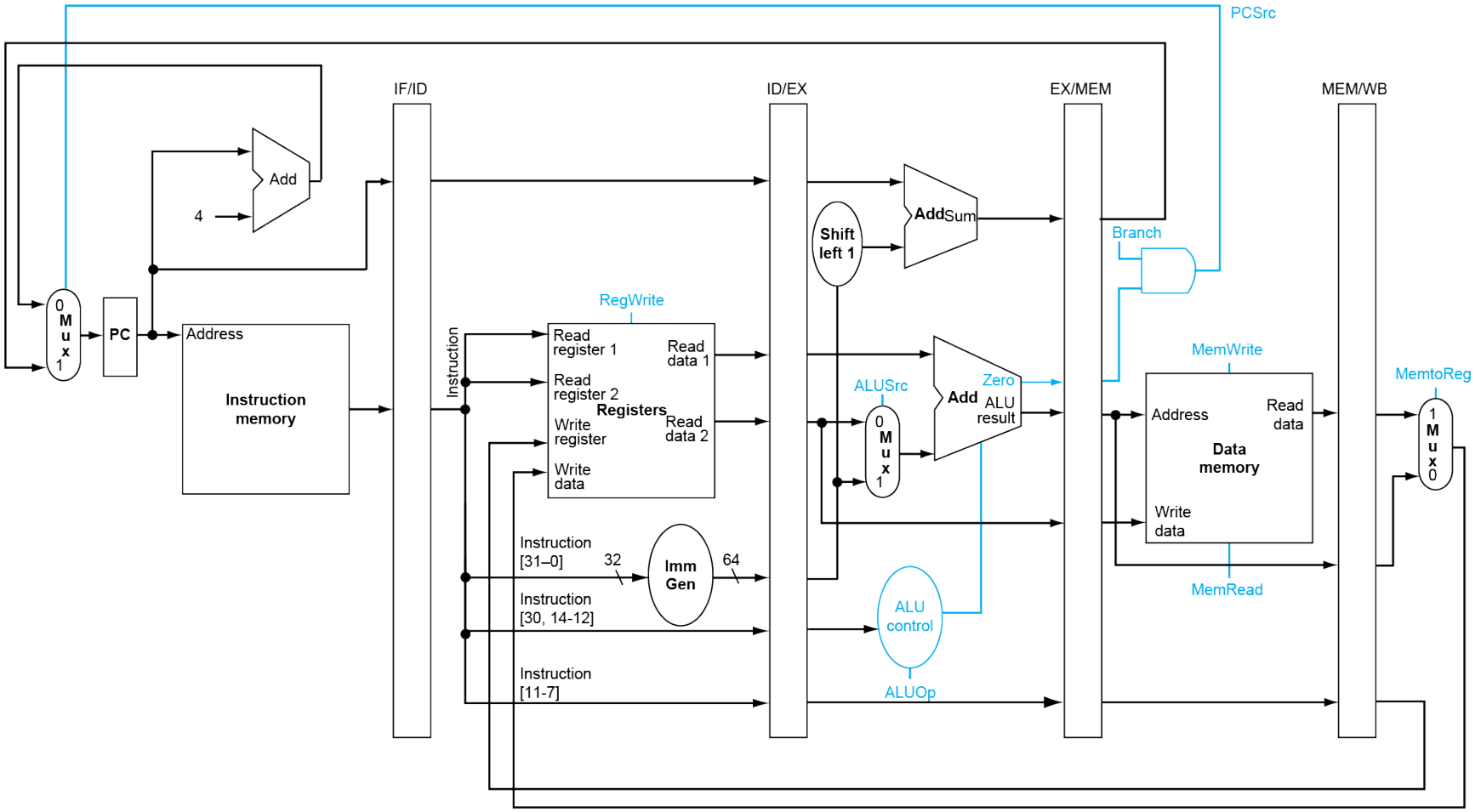


Revision: Multi-Cycle Diag

- Traditional form

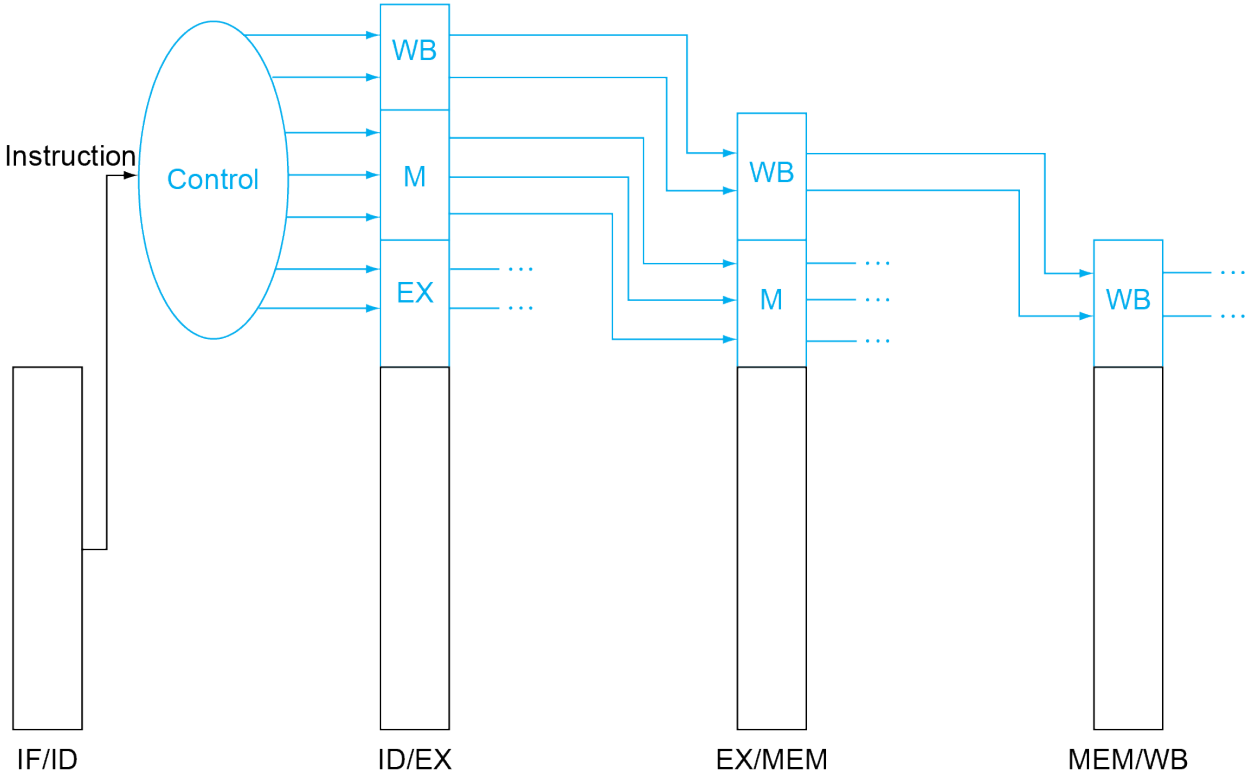


Pipelined Control (Simplified)

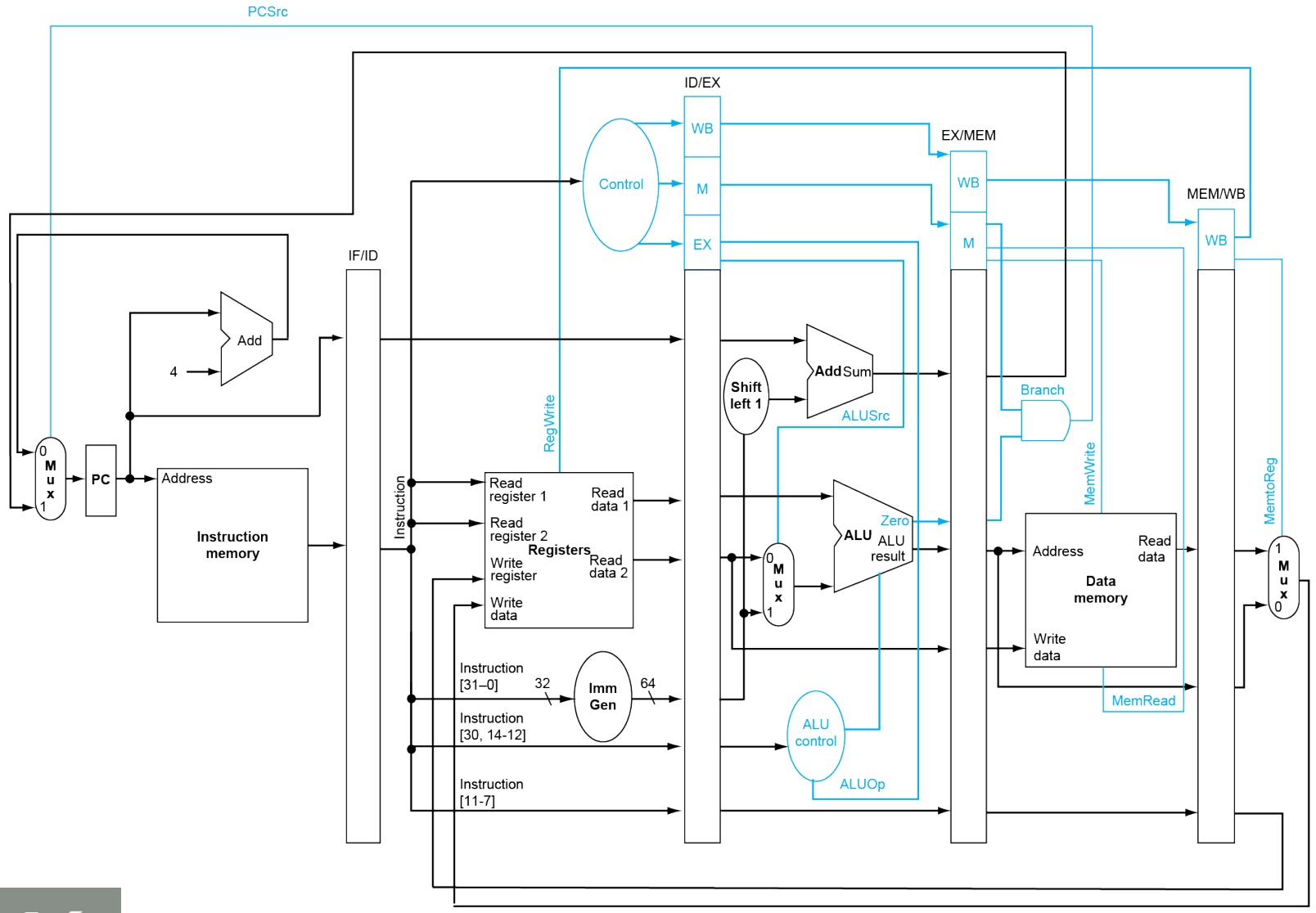


Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation

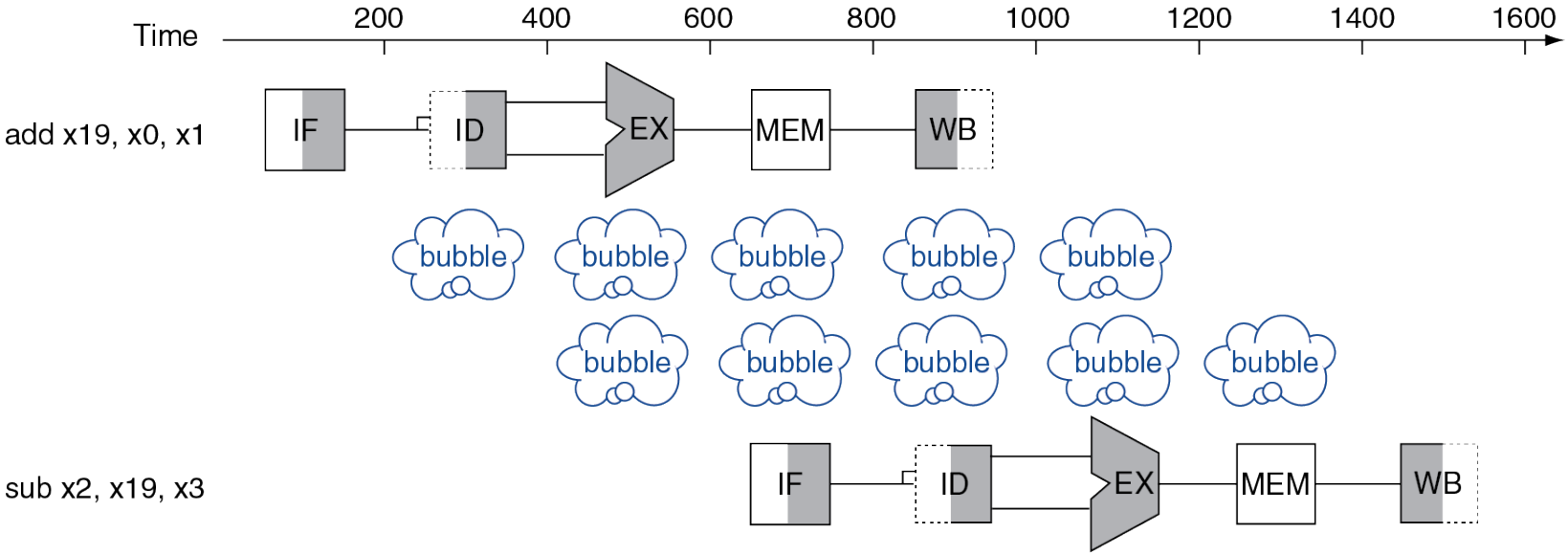


Pipelined Control



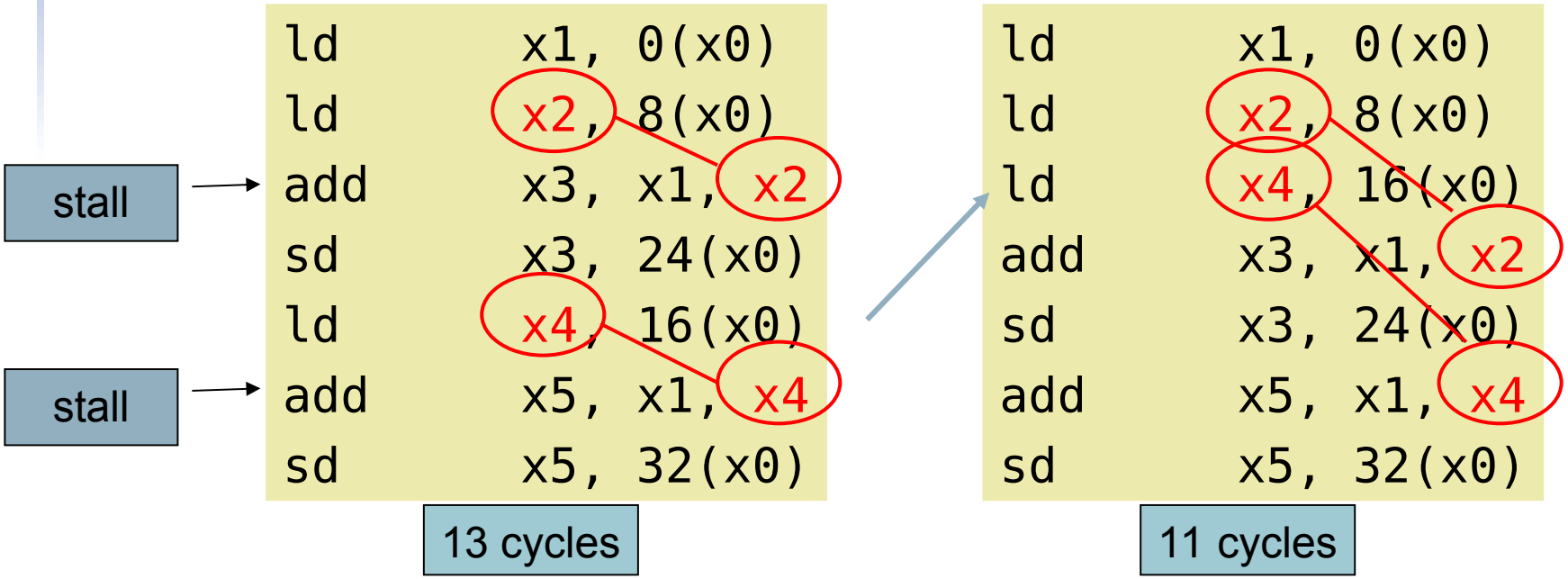
Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - `add x19, x0, x1`
    `sub x2, x19, x3`



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for `a = b + e; c = b + f;`



Data Hazards in ALU Instructions

- Consider this sequence:

```
sub  x2, x1,x3
and  x12,x2,x5
or   x13,x6,x2
add  x14,x2,x2
sd   x15,100(x2)
```

- We can resolve hazards with forwarding
  - How do we detect when to forward?

Dependencies & Forwarding